

Lec2: scanner (lexical analyzer)

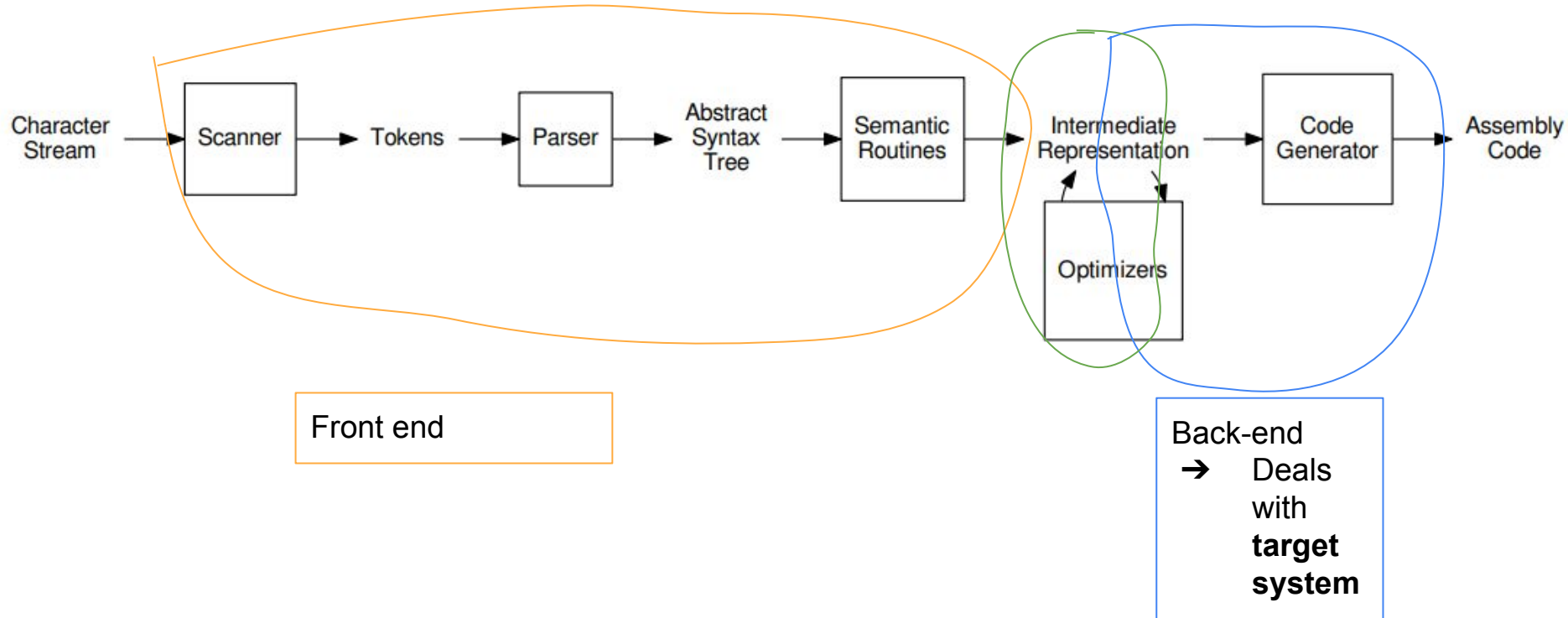
- Tokens, Regular Expressions & Finite Automata"
- AI connection: LLM tokenizers as lexers

Content is copied from:

- <https://web.stanford.edu/class/cs143/lectures/lecture03.pdf>
- <https://web.stanford.edu/class/cs143/lectures/lecture04.pdf>
- Engineering a Compiler by Cooper and Torczon, 2nd Ed. ch. 1 and sec. 2.1-2.4
- <https://www3.nd.edu/~dthain/compilerbook/chapter3.pdf>

Review: compiler structure

Scanner:
This week + next week



Scanner (lexical analyzer)

What do we want to do? Example:

```
if (i == j)
    z = 0;
else
    z = 1;
```

The input is just a string of characters:

→ `\tif (i == j)\n\t\tz = 0;\n\telse\n\t\tz = 1;`

→ `\tif (i == j)\n\t\tz = 0;\n\telse\n\t\tz = 1;`

Goal:

Partition this input string into meaningful substrings (tokens)

AI connection: This is exactly what an LLM tokenizer does — partition text into meaningful units. GPT's BPE tokenizer does the same job for natural language.

What is a token?

A syntactic **category**

– In English: noun, verb, adjective, ...

Token categories in any
Programming Language:

- Keywords
- Identifiers
- Numbers
- Strings
- Comments and whitespace
- ...

Scanner identifies **tokens** from the raw text
source code of a program.

What is a token?

Token categories in any
Programming Language:

Category	Examples
Keywords	if, else, while, return
Identifiers	x, count, myVar
Numbers	42, 3.14, 0xFF
Strings	"hello", 'world'
Operators	+, -, ==, <=
Punctuation	(,), {, }, ;
Comments/WS	// ..., /* ... */, spaces, tabs

In NLP, tokens are subwords or words. In an LLM, the "token categories" are vocabulary entries learned by BPE.

What are tokens for?

- Classify program substrings according to rules
- Lexical analysis produces a stream of tokens
 - ... which is input to the parser
- Parser relies on token distinctions
 - An identifier is treated differently than a keyword

In AI: token stream feeds into Transformer (not a parser, but the idea is identical — structured input for the next stage)

Designing a Lexical Analyzer (or designing a new language)

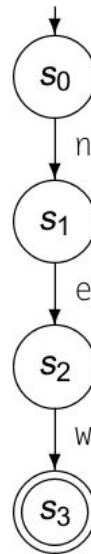
Step-1: Define a finite set of tokens

- Tokens describe all items of interest
 - Identifiers,
 - integers,
 - keywords(reserved words)
- Choice of tokens depends on
 - language
 - design of parser

Step-2: Describe which strings belong to each token

Recognizing words: A handmade scanner

```
c ← NextChar();  
if (c = 'n')  
  then begin;  
    c ← NextChar();  
    if (c = 'e')  
      then begin;  
        c ← NextChar();  
        if (c = 'w')  
          then report success;  
          else try something else;  
        end;  
      else try something else;  
    end;  
  else try something else;
```



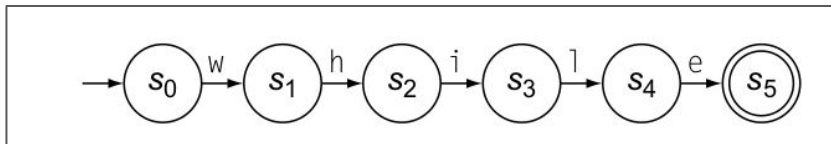
Transition
to error
states are
omitted.

■ **FIGURE 2.1** Code Fragment to Recognize "new".

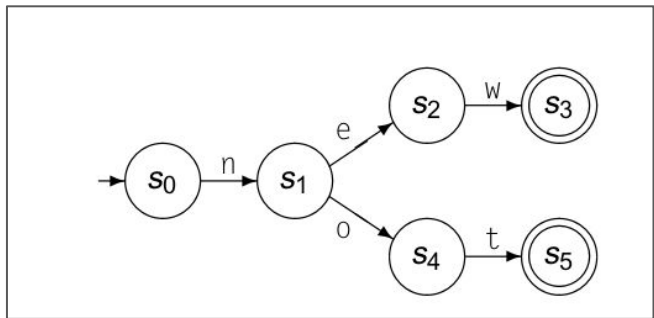
From the book Engineering a Compiler by Cooper and Torczon, 2nd Ed.

Recognizing words: Other examples

while

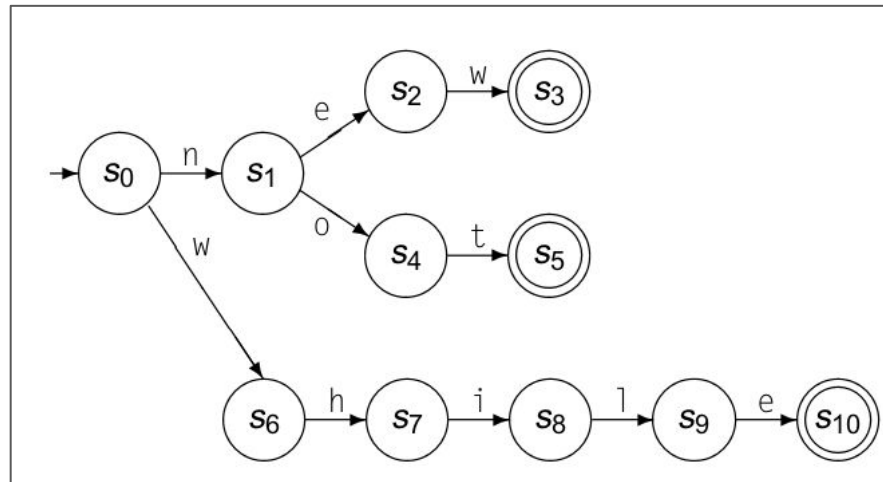


new and not



Combined version: while, new, not

- State s₀ has transitions for n and w



Handmade scanner in C

```
int scan_token(FILE *fp) {  
    int c = getc(fp);  
  
    if (c == 'w') {  
        if (getc(fp) == 'h')  
            if (getc(fp) == 'i')  
                if (getc(fp) == 'l')  
                    if (getc(fp) == 'e')  
                        return WHILE;  
        return ERROR;  
    }  
    else if (c == 'n') {  
        int c2 = getc(fp);  
        if (c2 == 'e') {  
            if (getc(fp) == 'w')  
                return NEW;  
        }  
        else if (c2 == 'o') {  
            if (getc(fp) == 't')  
                return NOT;  
        }  
        return ERROR;  
    }  
    else if (isdigit(c)) {  
        /* read number... */  
    }  
    /* ... more cases ... */  
    return ERROR;  
}
```

Is it as easy as it sounds?

Sort of... if you do not make it hard!

complications arise:

- **Lookahead:** `i` vs `if`, `=` vs `==`
- **Ambiguity:** Is `if` a keyword or two identifiers `i` and `f`?
- **Maximal munch:** How much input should I consume?
- **Error recovery:** What if no rule matches?

Natural language: No fixed alphabet, no reserved words, ambiguous boundaries → This is why LLMs use *learned* tokenizers (BPE) instead of regex

Historical Footnote: FORTRAN Whitespace

Lexical Analysis in FORTRAN

FORTRAN rule: Whitespace is insignificant

VAR1 is the same as VA R1

D05I is the same as DO 5 I

- ★ A terrible design!
- ★ Historical footnote: FORTRAN Whitespace rule motivated by inaccuracy of punch card operators

Historical Footnote: FORTRAN do-loops

Fortran do-loops

```
DO 5 I = 1,25      ← DO statement (loop)
```

```
DO 5 I = 1.25      ← Assignment to variable D05I
```

From left-to-right, it cannot tell if **DO5I** or **DO stmt** .

“**Lookahead**” may be required to decide where one token ends and the next token begins

- after “,” is reached, it can be determined.

Lookahead

Even our simple example has lookahead issues

– `i` vs. `if`

– `=` vs. `==`

The problems continue today: C++ Template Syntax

Lexical Analysis in C++

```
Foo<Bar>           // Template instantiation  
cin >> var;       // Stream extraction operator  
Foo<Bar<Bazz>>    // Is >> one token or two > tokens?
```

PL/I: Keywords Are Not Reserved

```
IF ELSE THEN THEN = ELSE; ELSE ELSE = THEN
```

- Cannot tell if DECLARE is a keyword or array reference until after). Requires **arbitrary lookahead!**
- These are **language design** problems that make the lexer's job harder.
 - many trade-offs in language design.

Goal of Lexical Analysis

- Partition the input string into lexemes
 - Identify the token of each lexeme
- ★ Left-to-right scan => lookahead sometimes required

We need

- A way to describe the lexemes of each token
- A way to resolve ambiguities
 - Is **if** two variables **i** and **f**?
 - Is **==** two equal signs **= =**?

Tokens vs. Lexemes vs. Patterns

Term	Definition	Example
Token	A syntactic category (type)	NUMBER
Lexeme	The actual string matched	"42"
Pattern	Rule describing all lexemes of a token	[0-9]+

```
Input:   x  =  42  +  y  ;  
Tokens:  ID  ASSIGN  NUMBER  PLUS  ID  SEMICOLON  
Lexemes: "x"  "="  "42"  "+"  "y"  ";"
```

A quick look for LLMs

	Traditional Lexer	LLM Tokenizer (BPE)
Input	Source code	Natural language
Rules	Hand-written regex	Learned from corpus
Ambiguity	Maximal munch + priority	Greedy longest merge
Unknown input	Error	Byte-level fallback

Regular Expressions

Regular Languages

There are several formalisms for specifying tokens

→ **Regular expressions** are the standard notation for **regular languages**:

- ◆ Simple and useful theory
- ◆ Easy to understand
- ◆ Efficient implementations (→ finite automata)

In AI/NLP, the equivalent "formalism" is not regex but statistical learning (BPE, SentencePiece). But the *goal* is the same: define which substrings are valid tokens.

Languages

Alphabet and Strings

- **Alphabet (Σ):** a finite set of characters/symbols
- **String (s):** a finite, possibly empty sequence of symbols from Σ
- **Language $L(s)$:** a set of strings drawn from Σ

Alphabet = English characters

Language = English sentences

★ Not every string of English characters is an English sentence

Alphabet = ASCII

→ Note: ASCII character set is different from English character set

Language = C programs

Regular expressions

Need some notation for specifying which sets we want

The standard notation for **regular languages** is **regular expressions**.

- **s** is a string
- **L(s)** is the “language of **s**.”

A regular expression s is a string which denotes **L(s)**, a set of strings drawn from an alphabet Σ .

Regular expression base case

- s is a string
- $L(s)$ is the “language of s .”

A regular expression s is a string which denotes $L(s)$, a set of strings drawn from an alphabet Σ .

Single character

- If $a \in \Sigma$ then,
 - a is a regular expression
 - and $L(a) = \{a\}$

ϵ is a regular expression

- $L(\epsilon)$ contains only the empty string, $\{\epsilon\}$

RE Base case summary

Expression	Language
a (where $a \in \Sigma$)	$\{a\}$
ϵ (epsilon)	$\{""\}$ (the empty string)
$\emptyset$	$\{\}$ (the empty set)

Regular expression built up rules

Union (alternation)

$s|t$ is a RE such that

$$L(s|t) = L(s) \cup L(t).$$

Different notation

$$A + B = \{s \mid s \in A \text{ or } s \in B\}$$

Example: $a \mid b \rightarrow \{a, b\}$

Concatenation

st is a RE such that

- $L(st)$ contains all strings formed by the concatenation of a string in $L(s)$ followed by a string in $L(t)$.

Different notation

$$AB = \{ab \mid a \in A \text{ and } b \in B\}$$

Example: $ab \rightarrow \{ab\}$

Closure(Iteration)

The Kleene closure of a set A , denoted A^*

$$A^* = \bigcup_{i \geq 0} A^i = A^0 \cup A^1 \cup A^2 \cup A^3 \cup A^4 \cup \dots$$

Example: $a^* \rightarrow \{\epsilon, a, aa, aaa, \dots\}$

Examples of closures

Closure(Iteration)

The Kleene closure of a set A, denoted A^*

$$A^* = \bigcup_{i \geq 0} A^i = A^0 \cup A^1 \cup A^2 \cup A^3 \cup A^4 \cup \dots$$

$$A^0 = \{\epsilon\}$$

$$A^1 = \{A\}$$

Expression	Language
$\{"ab", "c"\}^*$	$\{\epsilon, "ab", "c", "abab", "abc", "cab", "cc", "ababab", \dots\}$
$\{"a", "b", "c"\}^*$	$\{\epsilon, "a", "b", "c", "aa", "ab", "ac", "ba", "bb", \dots\}$
$\emptyset^*$	$\{\epsilon\}$

examples

Regular Expression	Language
<code>hello</code>	<code>{hello}</code>
<code>d(o i)g</code>	<code>{dog, dig}</code>
<code>moo*</code>	<code>{mo, moo, mooo, ...}</code>
<code>(moo)*</code>	<code>{ϵ, moo, moomoo, ...}</code>
<code>a(b a)*a</code>	<code>{aa, aaa, aba, aaaa, aaba, abaa, ...}</code>

Abbreviations and some properties

Notation	Meaning
$s?$	$s \mid \epsilon$ (zero or one)
s^+	ss^* (one or more)
$[a-z]$	$(a b \dots z)$
$[^x]$	$\Sigma - \{x\}$ (complement)
$[0-9]$	$(0 1 2 \dots 9)$

- Union: $A + B \equiv A \mid B$
- Option: $A + \epsilon \equiv A?$
- Range: $'a'+'b'+\dots+'z' \equiv [a-z]$
- Excluded range: complement of $[a-z] \equiv [^a-z]$

Associativity:	$a \mid (b \mid c) = (a \mid b) \mid c$
Commutativity:	$a \mid b = b \mid a$
Distribution:	$a(b \mid c) = ab \mid ac$
Idempotency:	$a^*a^* = a^*$

Also common in practice

- $.$ — any character
- $\{n\}$ — exactly n times
- $\{n,m\}$ — at least n , at most m times

exercises

`[0-9]+(.[0-9]+)?`

Which one matches?

- **123**
 - matches
- **3.14**
 - matches
- **.15**
 - Do not match
- **30.**
 - Do not match

`[A-Z]+([A-Z]|[0-9])*`

Which one matches?

- **PRINT**
 - matches
- **MODE5**
 - matches
- **hello**
 - Do not match
- **4YOU**
 - Do not match

`<[^>]*>`

Which one matches?

- **<tricky part>**
 - matches
- **<<<look left>**
 - matches
- **<this is an <illegal> comment>**
 - Does not match

examples

Regular Expression s	Language L(s)
[abc]⁺	
[abc][*]	
[0-9]⁺	
[1-9][0-9][*]	
[a-zA-Z][a-zA-Z0-9_][*]	

Tokens as Regular Expressions

Token	Regular Expression
Keywords	<code>"if" "else" "while" ...</code>
Integers	<code>[0-9]+</code>
Floats	<code>[0-9]+\.[0-9]+</code>
Identifiers	<code>[a-zA-Z_][a-zA-Z0-9_]*</code>
Whitespace	<code>[\t\n]+</code>
Comments (single-line)	<code>//[^\\n]*</code>
Phone number	<code>\([0-9]{3}\)-[0-9]{3}-[0-9]{4}</code>

Regular expression is used to describe grammars

Keywords

“else” or “if” or “begin” or ...

`'else' + 'if' + 'begin' + ...`

Abbreviation: `'else' = 'e' 'l' 's' 'e'`

Integers: an empty string of digits

```
digit = '0' + '1' + '2' + '3' + '4' + '5' + '6' + '7'  
+ '8' + '9'
```

```
integer = digit digit*
```

Abbreviation: $A^+ = AA^*$

Abbreviation: $[0-2] = '0' + '1' + '2'$

Identifier

Identifier: strings of letters or digits, starting with a letter

letter = 'A' + ... + 'Z' + 'a' + ... + 'z'

identifier = letter (letter + digit)*

Is **(letter* + digit*) = (letter + digit)*** ?

Whitespace

Whitespace: a **non-empty sequence of blanks, newlines, and tabs**

`(' ' + '\n' + '\t')+`

Numeric constants

digit ::= [0-9]

digits ::= digit⁺

number ::= digits (. digits)?

([eE] (+ | -)? digits) ?

Phone Numbers

Consider (650)-723-3232

Σ = digit + { -, (,), }

exchange = digit³

phone = digit⁴

area = digit³

phone_number = '(' area ')' '-' exchange '-' phone

Finite Automata:

From Specification to Implementation

Concept	Role
Regular expressions	Specification (what to match)
Finite automata	Implementation (how to match)

Recognizing REs

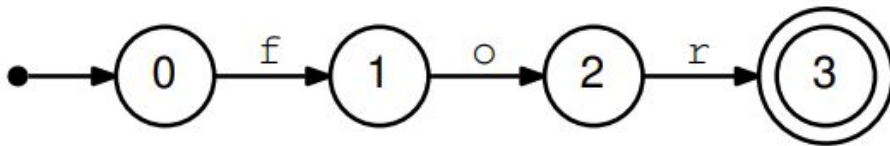
Given string **s** and
rexp **R**,

is $s \in L(R)$?

Finite automata can be used to recognize strings generated by regular expressions

Can build by hand or automatically

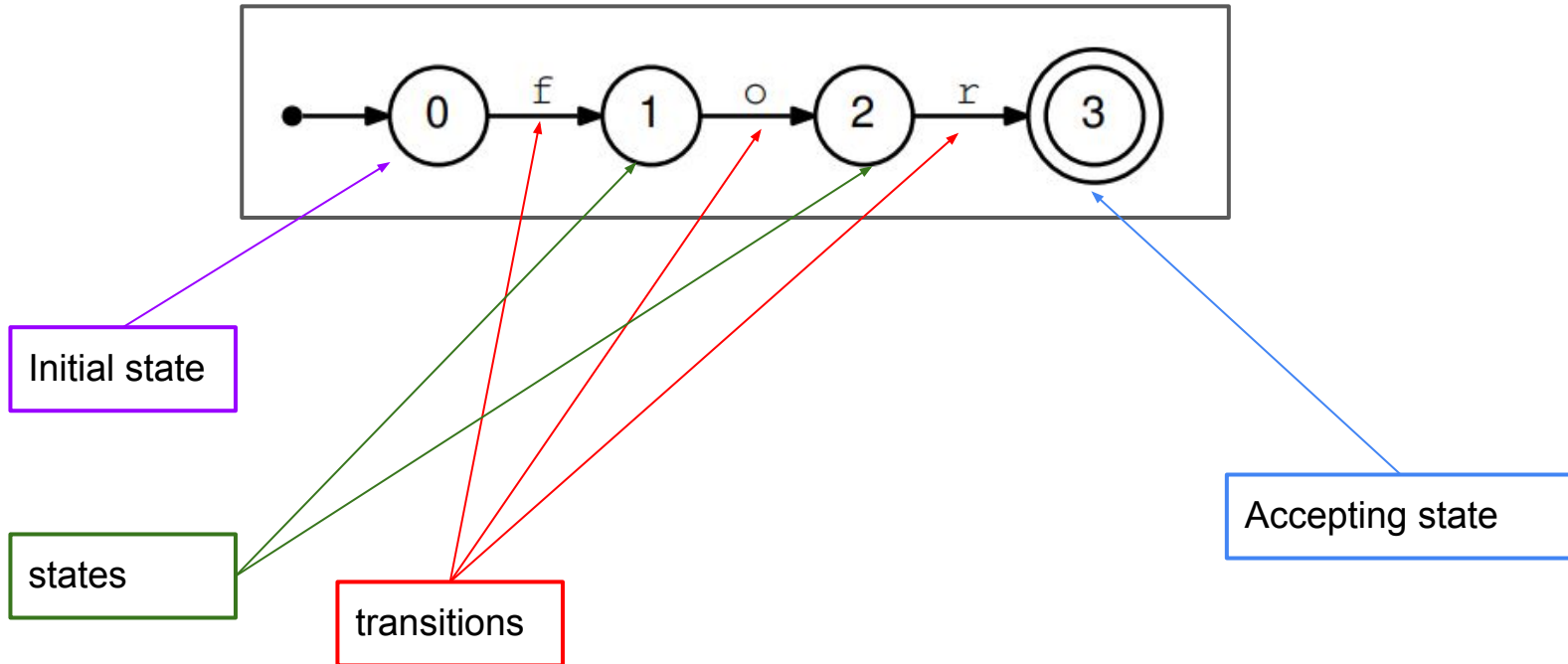
- Reasonably straightforward, and can be done systematically
- Tools like **Lex**, **Flex**, **JFlex** etc this automatically, given a set of REs.
- Same techniques used in **grep**, **sed**, and other packages/tools



In AI, the equivalent "tool" is the BPE training algorithm that learns the token vocabulary, then applies it deterministically at inference time.

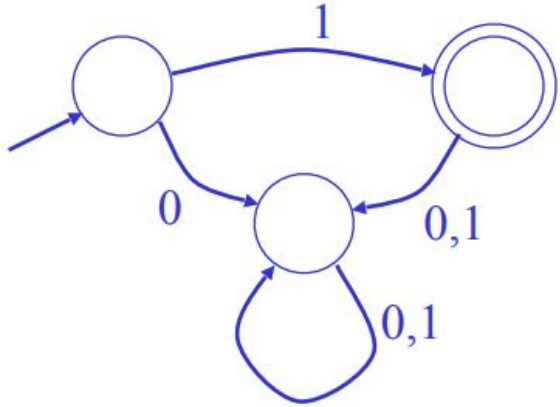
Finite Automata state graphs

A finite automaton (FA) is an abstract machine that can be used to represent certain forms of computation.



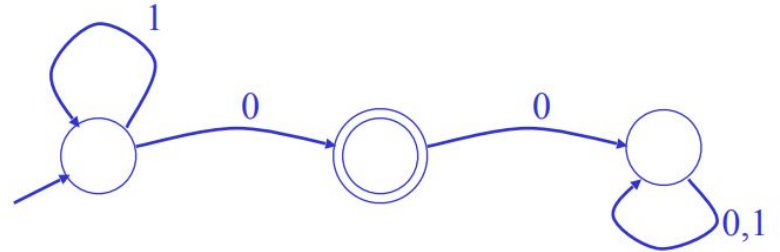
A simple example

A finite automaton that accepts only “1”



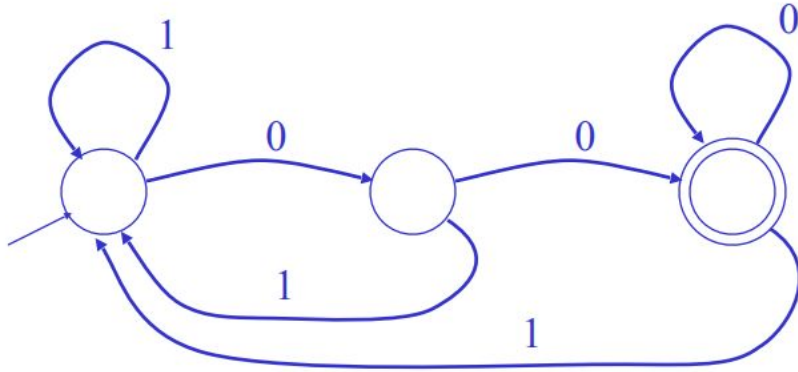
A finite automaton accepting any number of 1's followed by a single 0

Alphabet: $\{0,1\}$



Alphabet $\{0,1\}$

What language does this recognize?



More Examples

Example: `[a-z][a-z0-9]*`

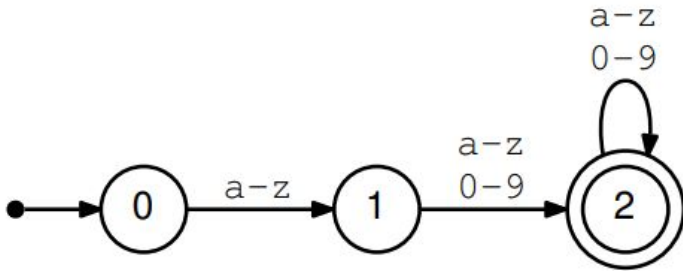
Example: `[1-9][0-9]* | 0`

Deterministic finite automata(DFA)

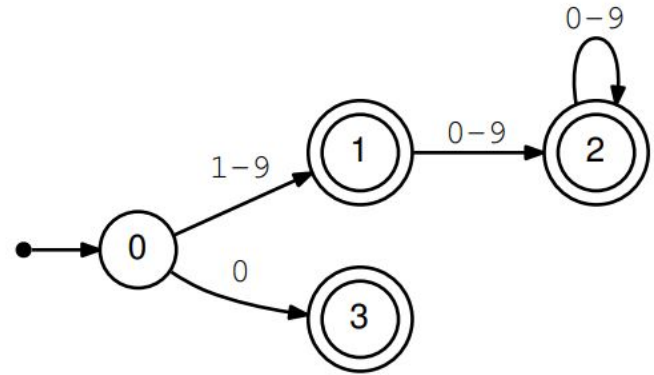
A **DFA** is a special case of an FA where:

- Every state has **exactly one** outgoing edge per input symbol
- **No ϵ -moves** (no transitions without consuming input)
- **No ambiguity:** for every (state, symbol) pair, exactly one next state

[a-z][a-z0-9]+



([1-9][0-9]*)|0



Deterministic finite automata(DFA)

Implementation:

- One integer c tracks current state
- Transition matrix $M[\text{state}][\text{input}]$ gives next state
- Very efficient: **$O(1)$ per character** (table lookup)
- **Error:** (If the transition is not allowed, we mark it with E to indicate an error.)

Acceptance:

For each symbol,

- We compute $c = M[s, i]$ until all the input is consumed, or an error state is reached.

DFA Implementation: Transition Table

```
M[s0]['a'] = s1
```

```
M[s0]['b'] = ERROR
```

```
M[s1]['b'] = s2
```

```
M[s1]['a'] = ERROR
```

```
M[s2]['a'] = ERROR ← no
```

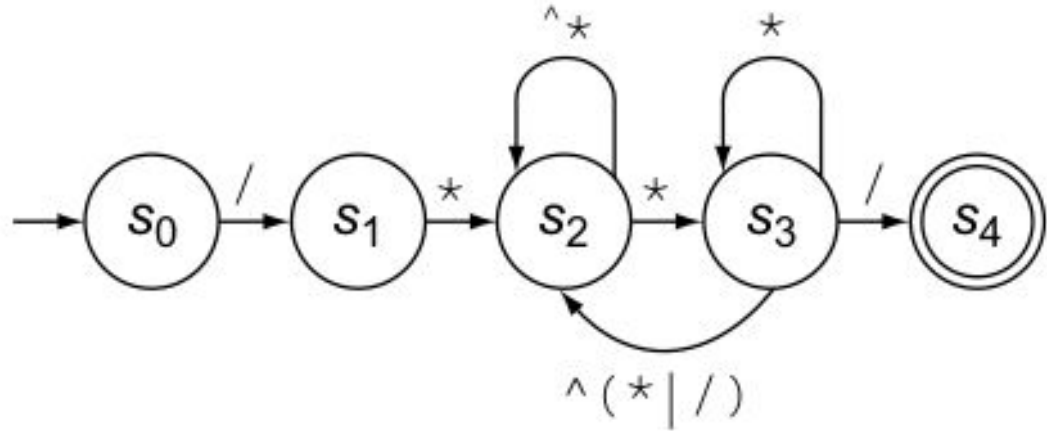
```
transition = error
```

```
int state = START;
while ((c = getchar()) != EOF) {
    state = M[state][c];
    if (state == ERROR) { report_error(); break; }
}
if (is_accepting(state)) { /* matched! */ }
```

RE for comments in C++, Java

`/* (^*)* */` without *

`/* (^* | *+^/)* */`

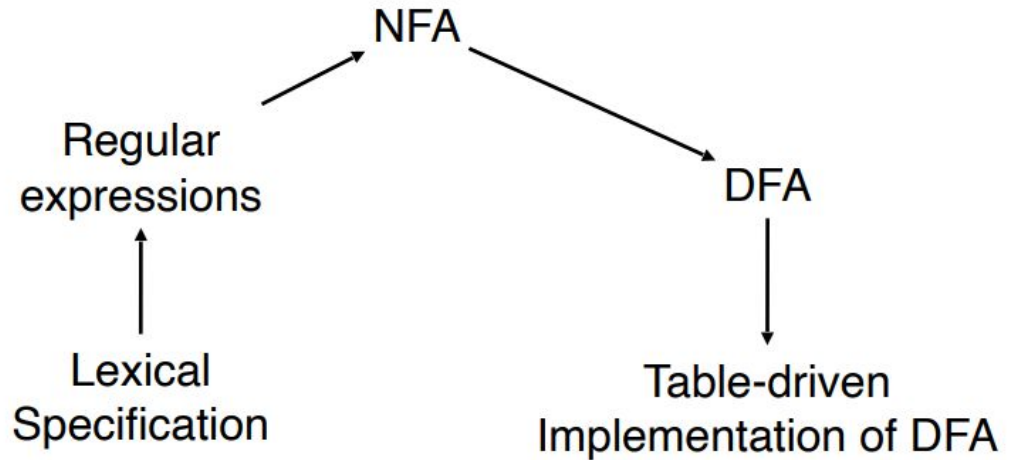


A more correct written form with escape char\ and without nested comments:

`/\* ([^*]|\\*+[^*/])*\*+\/`

Convert Regular Expressions to Finite Automata

High level sketch



Regular Expressions in Lexical Specification

a specification for the predicate

$$s \in L(R)$$

- **yes/no answer is not enough!**
- Instead: **partition the input into tokens**
- We will adapt regular expressions to this goal

Lexical Specification → Regex in five steps

1. Write a regex for each token

- `Number = digit +`
- `Keyword = 'if' + 'else' + ...`
- `Identifier = letter (letter + digit)*`
- `OpenPar = '('`
- ...

2. Construct R, matching all lexemes for all tokens

`R = Keyword + Identifier + Number + ...`
`= R1 + R2 + ...`

(This step is done automatically by tools like flex)

3. Let input be $x_1 \dots x_n$

for $1 \leq i \leq n$ check

$x_1 \dots x_i \in L(R)$

4. If success,

then we know that

$x_1 \dots x_i \in L(R_j)$ for some j

5. Remove $x_1 \dots x_i$ from input and go to (3)

Ambiguity

There are ambiguities in the algorithm

- How much input is used?

What **if** $x_1 \dots x_i \in L(R)$ **and** also $x_1 \dots x_k \in L(R)$

- Rule: Pick longest possible string in $L(R)$
 - Pick k **if** $k > i$
 - The “maximal munch”

Ambiguity

Which token is used?

What **if** $x_1 \dots x_i \in L(R_j)$ **and** $x_1 \dots x_i \in L(R_k)$

- Rule: use rule listed first
 - Pick **j** **if** $j < k$
 - E.g., treat "if" as a keyword, not an identifier

Error handling

What if **No rule matches a prefix** of input?

- **Problem:** Can't just get stuck ...
- **Solution:**
 - Write a rule matching all "bad" strings
 - Put it **last** (lowest priority)

Error handling

Regular expressions provide a concise notation for string patterns

- Use in lexical analysis requires small extensions
 - To resolve ambiguities
 - To handle errors
- Good algorithms
 - Require only single pass over the input
 - Few operations per character (table lookup)

Finite automata again

Regular expressions = specification

Finite automata = implementation

Deterministic Finite Automata (DFA)

- Exactly one transition per input per state
- No ϵ -moves

Nondeterministic Finite Automata (NFA)

- Can have zero, one, or multiple transitions for one input in a given state
- Can have ϵ -moves

AI Connection: LLM Tokenizers as Lexers

Traditional Lexer (Flex)	LLM Tokenizer (BPE)
Input: source code	Input: natural language text
Output: token stream	Output: token ID sequence
Rules: regex (hand-written)	Rules: learned from data
Vocabulary: ~50-200 tokens	Vocabulary: 32K-128K tokens
Ambiguity: maximal munch + priority	Ambiguity: greedy longest merge
Unknown char → error	Unknown → byte-level fallback
Tool: Flex, PLY, OCamllex	Tool: SentencePiece, tiktoken, HF Tokenizers

Both solve the same problem: partition a string into meaningful units. The difference is *how* the units are defined.

Byte Pair Encoding (BPE): How it works

Training algorithm:

- Start with individual characters (or bytes) as vocabulary
- Count all adjacent pairs in the corpus
- Merge the most frequent pair into a new token
- Repeat until desired vocabulary size

Corpus: "low low low lowest"

Initial: l o w l o w l o w l o w e s t

Step 1: merge (l,o) → "lo" [most frequent pair]

Step 2: merge (lo,w) → "low"

Step 3: merge (e,s) → "es"

Step 4: merge (es,t) → "est"

...

At inference: Apply merges in order (greedy longest match) — deterministic, just like a DFA!

Regex vs BPE

input: "let x = 42;"

Regex

Tokens: [LET, ID("x"), ASSIGN, NUMBER(42), SEMI]

Count: 5 tokens

GPT-4 BPE (tiktoken):

Tokens: ['let', ' x', ' =', ' 4', '2', ';']

Count: 6 tokens

- BPE splits 42 into 4 and 2 (subword level)
- BPE preserves whitespace as part of tokens (' x', ' =')
- Flex produces *semantic* tokens; BPE produces *statistical* tokens

Why Not Just Words or Characters?

Approach	Problem
Word-level	"unhappiness" → unknown token; vocabulary explodes
Character-level	"unhappiness" → 12 tokens; too many, loses semantics
Subword (BPE)	"unhappiness" → ["un", "happiness"] — 2 tokens ✓

BPE is the middle ground:

- Handles rare words by decomposition
- Vocabulary stays manageable (32K-128K)
- Deterministic at inference (like a DFA!)

Discussion Questions

1. Why can't we use regex for natural language tokenization?
 - Hint: ambiguity, context-dependence, infinite vocabulary
2. BPE is "greedy longest match" — how is this similar to maximal munch?
3. Why do LLM tokenizers include whitespace as part of tokens (e.g., " the" vs "the")?
4. If you were designing a programming language, what token design choices would make the lexer's job harder?
 - Think: FORTRAN whitespace, C++ >>, PL/I keywords